

PROBLEM STATEMENT - 1

AGRISMART AI

Intelligent Agriculture for a Sustainable Future

Domain	Technology	Target Users	Challenge Level
AI / AgriTech / Sustainability	AI/ML • Computer Vision • GenAI • (IoT optional)	Farmers & Agricultural Stakeholders	Advanced (above-average difficulty)

1. Background

Agriculture faces persistent challenges: water scarcity, unpredictable weather, soil degradation, crop diseases, inefficient irrigation, and the excessive use of fertilizers and pesticides. Emerging technologies - artificial intelligence (AI), computer vision, satellite and weather data, and predictive analytics - can help farmers make better, more sustainable decisions. This challenge asks you to turn one such capability into a working, verifiable tool, and gives you room to extend it into a broader advisory platform.

2. Core Problem Statement

Build an AI-powered crop-disease detection tool that identifies plant diseases from leaf/crop images using computer vision, reports its accuracy on a held-out test set we provide, and presents results to a farmer in a simple, actionable way. You may then extend it into a broader smart-agriculture advisor by adding any of the optional bonus modules in Section 3.2.

3. Challenge Structure: Core + Bonus

3.1 Mandatory Core Task (required of every team)

- **Crop Disease Detection (Computer Vision).** Accept a leaf/crop image and classify it into a disease class (or “healthy”) from the provided label set. Report standard metrics on the provided held-out test set (Section 4), and surface a clear result plus basic precautionary guidance to the user.
- **Minimum bar.** A trained model (transfer learning is encouraged), an honest train/validation/test split, a reported macro-F1 and confusion matrix on the held-out test set, and a minimal interface (web app, notebook UI, or CLI + screenshots) that runs a prediction on a new image.

A submission that does not deliver the core task is considered incomplete regardless of how many bonus modules it includes.

3.2 Optional Bonus Modules (each adds points - build any, all, or none)

Bonus Module	What it adds
A. Crop Recommendation	Recommend suitable crops from soil type, pH, temperature, humidity, rainfall, water availability, season, location, and previous crop. Report your metric and data source.
B. Smart Irrigation	Predict whether irrigation is required from soil moisture, weather forecast, crop type, and growth stage. State the logic or model and how it was validated.
C. Weather-Based Intelligence	Combine live/forecast weather with farm conditions to produce actions (e.g. “delay irrigation - rain likely”, “raised disease risk - monitor”). Name your weather data source.
D. Sustainability Score	Compute an indicative score from water efficiency, resource use, and crop health, with improvement suggestions. Publish the exact formula/rules so it is reproducible.
E. Farmer Assistant (GenAI)	A conversational or voice interface that explains recommendations in plain language, ideally with regional-language support. Grounded answers score higher than free-form generation.
F. IoT Integration	Integrate real sensor data (soil moisture, temperature, humidity, pH). Hardware is NOT expected online - a documented simulated/streamed sensor feed is fully acceptable and scored equally.
G. Agentic Advisor	An autonomous agent that continuously analyses inputs, reasons, checks weather/data, decides, and notifies the farmer. Show the decision loop and at least one end-to-end automated recommendation.

4. Provided Data & Evaluation Protocol

4.1 The dataset & the deliberate difficulty

The core task uses two well-known public datasets in a train-on-lab / test-in-field design. This is intentional: it is what makes the challenge above-average. Models that memorise clean laboratory images score in the high-90s on other lab images but degrade sharply on real field photos - so the real problem is generalisation, not fitting.

	Training / validation	Held-out test
Source	PlantVillage (lab-condition leaf images, uniform background).	A field-condition set derived from PlantDoc-style real-world images (natural lighting, clutter, occlusion).
Scale	~54,000 images across the shared class list, released at kickoff.	A separate, unseen set used only during judging; a small public sample is shared at kickoff for format checks.
Role	Train and validate your model here. You may add other public data (cite it).	Never train on it. Your reported core metric is computed here by organizers.

- **Shared class list.** A fixed set of ~15–20 crop–disease classes (plus “healthy”) that exist in BOTH datasets is published with the data, e.g. Tomato Early Blight, Tomato Late Blight, Tomato Leaf Mould, Tomato Bacterial Spot, Potato Early/Late Blight, Corn Common Rust, Corn Grey Leaf Spot, Apple Scab, Apple Black Rot, Grape Black Rot, Bell Pepper Bacterial Spot, and the matching “healthy” classes. The exact final list ships with the kickoff data; all teams use it verbatim.
- **Submission / predict interface.** Expose a `predict(image_path) → class_label` function (Python) OR a documented CLI (`python predict.py --image path`) that prints the predicted class. It must load your trained weights and run on a single new image with no manual steps. Exact signature and the label-string format are published with the data.

4.2 Primary metric (core task)

- **Macro-averaged F1** on the held-out test set is the primary, objective ranking metric for the core task. Macro-F1 is used (not raw accuracy) because disease classes are imbalanced.
- You must also report a confusion matrix and per-class precision/recall in your model report (Section 7.3).

Data rules for the core task.

You may add extra public data to your TRAINING set (cite it) and use any data you like for bonus modules. But the core task is always evaluated on the organizers' held-out field set via your predict interface. Training on the held-out set, or substituting your own test data for the core score, is non-compliant and forfeits the AI/ML axis.

5. Example User Scenario (illustrative)

A farmer photographs an affected leaf and enters basic context. The tool classifies the disease and - if bonus modules are present - combines this with weather and farm data to produce clear, farmer-friendly guidance.

Input	Example Value
Leaf image	Uploaded photo of affected leaf
Crop / Stage	Tomato / Growing
Soil / pH / Moisture	Loamy / 6.5 / 31%
Temperature / Rain probability	28–32°C / High

Sample AI Output

- **Disease detection (core):** Early Blight - confidence 0.91. Precaution: remove affected leaves, avoid overhead watering.
- **Irrigation (bonus B):** Delay irrigation - rainfall likely in next 24h.
- **Sustainability insight (bonus D):** Estimated reduction in unnecessary water use this week.
- **Assistant (bonus E):** Plain-language explanation, optionally in a regional language.

6. AI / Technology Expectations

Technology	Where it fits
Computer Vision (core)	Leaf/crop image classification for disease detection. Transfer learning (e.g. a pretrained CNN/ViT backbone) is encouraged.
Machine Learning	Crop recommendation, irrigation prediction, yield/health forecasting (bonus modules).
Generative AI	Plain-language explanations and Q&A. Grounded, cited answers are preferred over free-form text.
Predictive Analytics	Weather-based risk analysis and resource forecasting.
IoT (optional)	Simulated or real sensor feeds for soil moisture, temperature, humidity, pH.

7. Submission Contract (GitHub)

All teams submit a single public GitHub repository link. A submission is judged only if it meets the requirements below. Reproducibility is a scored gate: if judges cannot run or verify your core task, the AI/ML axis cannot be awarded in full.

7.1 Required repository structure

- **/README.md** - the entry point (see 7.2).
- **/src or /app** - source code.
- **/model** - training/inference code and the required predict interface from Section 4.1 (weights via release/link if large).
- **/report** - the one-page model report (7.3).
- **requirements.txt / environment file** and clear run instructions.

7.2 Required README contents

1. Which core + bonus modules you built.
2. Setup and run instructions (a judge must reproduce a prediction in under ~10 minutes).
3. Dataset used and its source/licence (core = provided dataset).
4. Reported metrics for every model (core: macro-F1 + confusion matrix).
5. Architecture overview and known limitations.
6. Link to the demo video (7.4) and to any deployed app.

7.3 Required model report (one page)

Field	What to state
Task	e.g. crop-disease image classification, N classes.
Dataset & split	Source, sizes, and exact train/validation/test split (core uses the provided split).
Model / approach	Architecture, backbone, key hyperparameters.
Metric & result	Macro-F1 (primary) + accuracy, with confusion matrix and per-class precision/recall.
Baseline	The provided baseline number and how you compare to it.
Limitations	Honest failure cases (e.g. real-field vs lab images).

7.4 Demo video (primary evidence)

- A **3–5 minute recorded demo** (unlisted link is fine) is the primary demonstratio. It must show the core task running on a new image, plus any bonus modules.

8. Originality & Timeframe Rules

- **All substantive work must be committed between 10 – 15 September.** Commit history must reflect real development during the window. Repos created before kickoff with pre-built solutions are disqualified.
- Reuse of open-source libraries, pretrained backbones, and public datasets is allowed and encouraged - but must be cited in the README.
- **Copying a public notebook/solution wholesale is prohibited.** Include an originality declaration in the README listing any third-party code or notebooks you referenced.
- AI coding assistants may be used; the working system and its evaluation are what get scored.

9. Evaluation Criteria (100 points)

Parameter	Weight	Scoring anchors
AI/ML Implementation	25	Primarily the held-out macro-F1 on the field test set, mapped to published bands anchored to the baseline (e.g. below baseline = low; baseline+0.10 = mid; well above baseline = high), plus soundness of method (honest split, no leakage, correct metric reporting). Strong bonus models add here.
Technical Implementation	20	Reproducibility (does it run from the README?), code quality/structure, integration of modules, and deployment. This axis is gated: unreproducible core = capped score.
Innovation & Creativity	15	12–15 = novel approach or genuinely new capability; 7–11 = thoughtful extension of known methods; 0–6 = standard tutorial-level solution.
Sustainability & Social Impact	15	12–15 = quantified resource/impact benefit with a stated method; 7–11 = credible qualitative impact tied to features; 0–6 = impact asserted but not substantiated.
User Experience	10	Farmer-friendliness, clarity of recommendations, accessibility /regional-language effort, and overall usability shown in the demo.
Problem Understanding	10	Clarity of problem framing, appropriateness of chosen modules, and honest limitations in the report.
Presentation & Demo	5	Clarity and completeness of the 3–5 min video (and live demo for the shortlist).
TOTAL	100	

10. Tie-Break & Shortlisting

To produce a clean top-44 cut, ties and near-ties are resolved in this fixed order:

1. **Core-task macro-F1** on the held-out test set (higher wins).
2. **Reproducibility** - did the core run cleanly from the README?
3. **Depth and correctness of bonus modules** (verified, not merely present).
4. **Innovation score**, then judge panel review of the demo.

11. Innovation Opportunities

Ideas that historically differentiate strong teams (all optional):

- Robustness to real-field images (not just clean lab photos)
- Voice-based farmer assistant
- Regional-language support
- GPS-based farm mapping
- Early warning for crop stress
- AI-based crop-rotation recommendation
- Water-consumption / yield prediction

- Carbon-impact estimation
- Offline / low-connectivity mode
- Agentic AI for autonomous advisory

12. Optional IoT Architecture (bonus only)

Only relevant to teams attempting bonus module F. Physical hardware is not expected in an online hackathon - a documented simulated sensor feed is scored equally.

Sensors (real or simulated) → ESP32 / Raspberry Pi → Cloud / API → AI/ML Engine → Recommendation Engine → Web / Mobile App → Farmer

13. Final Challenge Statement

AgriSmart AI - Intelligent Agriculture for a Sustainable Future. Build an AI-powered crop-disease detection tool (the core task) that classifies plant diseases from images and reports its accuracy on the provided held-out test set, presented in a simple, farmer-friendly way. Optionally extend it into a broader advisor - crop recommendation, smart irrigation, weather intelligence, a sustainability score, a conversational/regional-language assistant, IoT, or an agentic layer. Submissions are judged on a shared, objective core metric plus the depth, reproducibility, and impact of what you build, to create an affordable and scalable solution with measurable benefit to agricultural productivity and sustainability.